

Segment - 3

Relational Algebra:

- It is a procedural language of database
- It take one or more table or relation as input and produces one relations as output.
- Two types of operations under Relational Algebra.

- Primary (query ko select, from always chahiye)
- Secondary (query ko where always chahiye
na, ya phir condition diya jayega where
chahiye)

- Projection - Relational Algebra ko π (column select)
- Selection - " " " σ (specific row select)

SQL

select column-name
from table-name;

Relational Algebra:

$\pi_{\text{column-name}} (\text{table-name})$

Display first-name, job-id, employees.

⇒ SQL select first-name, job-id

Relational Algebra

Find the first name, job-id of those employees whose job-id is IT-PROG

from employees

Relational Algebra:

$$\pi_{\text{first-name, job-id}} \left(\sigma_{\text{job-id} = \text{'IT-PROG'}} (\text{employees}) \right)$$

এখানে Relational Algebra ত select এর স্যাক্রিফট
 π - হল ~~con~~ σ এর, where এর স্যাক্রিফট σ (sigma)
 - হল σ - এর table name σ বসাবার - হল And
 condition sigma এর নিচে হল।

Select * (All) কোনো * চিহ্ন দ্বারা All বুঝায়।

Find the employees whose salary is greater than 1500.

⇒ select first_name, job_id, salary
from employees

where salary > 1500

Relational Algebra:

π first_name, job_id, salary $\left(\sigma \text{ salary} > 1500 \right)$ (employees)

Logical Conditions

(AND, OR, NOT)

Relation Algebra to,

AND - $\wedge$

OR - $\vee$

NOT - $\neg$

select the row if an employee is a president and earns more than \$15000 or if the employee is a sales representative.

⇒ select first_name, salary, job_id
from employees

where job_id = 'AD-PRES' and salary > 15000

job_id = 'SA-REP';

Relation Algebra:

$\pi_{\text{first_name}, \text{job_id}, \text{salary}}$ (employee
job_id = 'AD-PRES')

and salary > 15000 Job_id = 'S' on job_id = 'SA-REP')

Cartesian join:

Join is a special form of cross product of two tables.

In Cartesian product we join a tuple of one table with the tuples of the second table. But in join there is a special requirement of relationship between tuples.

SQL Joins:

An SQL JOIN clause combines rows from two or more tables. It creates a set of rows in a temporary table.

Cartesian join:

প্রত্যেকটি row প্রত্যেকটি row এর সাথে connect হবে।

Types of join:

- Inner join
- Outer join
- Cross Join/ cartesian Join

Inner Join: Returns all rows from both the participating tables where the key record of one table is equal to the key records of another table.

Outer Join:

Outer joins are joins that return matched values and unmatched values from either or both tables.

Cross Join:

It is also known as CARTESIAN JOIN which returns the Cartesian product of two or more joined tables. The cross join produces a table that merges each row from the first table with each second table row. It is not required to include any condition in cross join.

tuple $\rightarrow$ row

relation $\rightarrow$ table

requirement आशा साफ
condition आशा

Syntax:

select * from table_1 cross join table_2;

Inner Join:

will return those tuples which will be matching the condition

- Equi Join
- Natural Join

Equi Join:

An equi-join is a join based on equality or matching column values. This equality indicated with an equal sign (=) as the comparison operator in the 'where' clause as the following query shows:

```
select *  
from manufact, stock  
where manufact.manu_code = stock.manu_code;
```

SQL Equi Join:

SQL Equi Join performs a Join against equality or matching column(s) values of the associated tables. An equal sign(=) is used as comparison operator in the where clause to return

equality.

You may also perform EQUI Join by using Join keyword followed by ON keyword and then specifying names of the columns along with their associated tables to check equality.

id	name	class	city
3	Hina	3	Delhi
4	Mgha	2	Delhi
6	Grouni	2	Delhi

Table name - Record

In this table, you have I'd, class and city are the fields.

Select *
from record;

What is Natural Join in SQL?

⇒ We have already learned that an EQUI Join performs a Join against equality or matching columns (s) values of the associated tables and an equal sign (=) is used as comparison operator in the where clause to refer equality.

The SQL NATURAL JOIN is a type of EQUI JOIN and is structure in such a way that columns with the same name of associated tables will appear once only.

Difference between natural join and equi join

⇒ Natural join ~~is~~ automatically join two tables on columns which have equal name and in the ~~next~~ result, the common column would appear one time, but in equi join we explicitly mention the condition that on which column we have to do equality test and also the columns would appear ~~wise~~ twice.

Cartesian Cross Join:

SQL:

```
select sid, sname, tname  
from student cross join teacher
```

RA:

$\pi_{(sid, sname, tname)} (student \times teacher)$

Find the name of those students who register the course CSE-2323

⇒ Select sid, sname, cid
from student, registers

where student.sid = registers.sid and cid = 'CSE-2323';

RA:

⋈ registers, sid, sname, cid (student.sid = registers.sid
(student ⋈ registers)
cid = 'CSE-2323')

Natural Join:

RA:

⋈ registers, sid, sname, cid (cid = 'CSE-2323' (student ⋈ registers))

Outer Join:

left outer join ⋈
Right outer join ⋈
full outer join ⋈

Exam left outer join:

Student	
Sid	Sname
S01	X
S02	Y
S03	Z

Register	
Sid	Cid
S01	C01
S02	C02
S03	C01

SQL:

select student.sid, sname, cid

from student left outer join register

ON student.sid = register.sid

RA:

student, sid, sname, cid $\left(\sigma_{\text{student.sid} = \text{register.sid}} (\text{student} \bowtie \text{register}) \right)$

Fundamental operators used in RA:

1) Selection (σ) - allows us to retrieve specific records or rows from a relation based on certain conditions.

2) Projection (π) - It displays the specific column of a table.

③ Union (U)

Must have same Same datatype Duplicate row will be eliminated

④ Set difference (-)

takes the two set and returns the value that are in the first set but not the second set.

⑤ Intersection (∩)

return common row

⑥ Rename (ρ)

is one of the unary operators in relational algebra and is used to rename relations in a DBMS

⑦ Cartesian product (×)

an operator used to merge columns from two relations

Union:

$\pi_{sid}(\text{student}) \cup \pi_{sid}(\text{register})$

Find student who has registered 'CSE - 001'

⇒ SQL:

select student.sid, sname, eid

from student register
 where student.sid = register.sid and cid = 'C01'.

RA:

$\pi_{s.sid, sname, cid} \left(\left(\pi_{s.sid}(\text{register}) \times \pi_{sname}(\text{student}) \right) \wedge s.sid = r.sid \wedge cid = 'C01' \right)$

Student (sid, sname, age)
 register (sid, cid)

course (cid, cname, credit)

Find the students and course details of all student who have registered.

⇒

Student		
sid	sname	age
S01	X	20
S02	Y	20
S03	Z	20

register	
sid	cid
S01	C01
S02	C02

Course		
cid	cname	Credit
C01	CSE-2323	3
C02	CSE-1211	3

SQL:

select s.sid, sname, age, r.cid, cname
from student s, register r, course c
where s.sid = r.sid and r.cid = c.cid

Join:

select s.sid, sname, age, r.cid, cname
from student s join register r
ON s.sid = r.sid Join course c
ON r.cid = c.cid

RA: (natural)

$\pi_{s.sid, sname, age, r.cid, cname} (P_s(student) \bowtie P_r(register) \bowtie P_c(course))$
On,

$\pi_{s.sid, sname, age, r.cid, cname} (P_s(student) \bowtie P_r(register) \bowtie P_c(course))$
 $\left(\begin{array}{l} s.sid = r.sid \wedge r.cid = c.cid \end{array} \right)$

Generalized:

Find the $\bowtie$ a process in which a new entity is formed using the common attributes of two or more entities.

Find the employees with increased salary

⇒ SQL:

```
select first_name, salary + 100  
from employees;
```

RA:

$\pi_{\text{first_name}, \text{salary} + 100}(\text{employees})$

Generalized Projection:

It extends the projection operation by allowing functions of attributes to be included in the projection list.

Division operator:

The division operator is used for queries which involve the 'all'.

Aggregate function:

2 ways to do the operation —
Group wise
Total wise

The functions are: AVG, MIN, MAX, COUNT, SUM

Find the total salaries of all employees together.

⇒ select SUM(salary)
from employees } Total wise

Group wise:

select job-id, SUM(salary)
from employees

Group by job-id

select e-id, COUNT(employee)
from employees

Group by department-id;

RA

department-id (employee)

select job-id, SUM(salary)
from employees
group by job-id;

RA:

job-id SUM(salary) (employees)

Sub Queries:

A sub query is a query that appears inside another query statement.

Find the details of all employees whose salary is same as 'lex'.

⇒ Select *

from employees

where salary = (select salary
from employees
where first_name = 'lex' or
last_name = 'lex')

RA

σ_{salary} (⋈_{salary} (σ_{first_name = 'lex' ∨ last_name = 'lex'} (employees)))

Character manipulation function:

CONCAT("Hello", "world")

SUBSTR("Hello", 1, 3)

LENGTH('Hello')

INSTR('Hello', 'el')

L PAD (salary, 5, '*')

R PAD (salary, 5, '*')

Display employees first name & last name joined together, the length of the employees last name and numeric position of the letter a in the employee last name for all employees who have the string REP contained in the job-id starting at the fourth position.

⇒ select CONCAT(first_name, last_name) 'Name',
LENGTH(last_name), INSTR(last_name, 'a')
from employees
where SUBSTR(job_id, 4) = 'REP'